

Лекция 5. Bash-автоматизация и анализ системных данных

Цель лекции

Понять базовую структуру административного Bash-сценария

Научиться использовать переменные, функции, аргументы и условия

Собрать рабочий `server-health.sh` для проверки диска и `systemd`-службы

Возвращать понятные `exit codes` для OK, WARNING, CRITICAL и UNKNOWN

Проверять сценарий перед запуском и готовить его к безопасному повторному использованию

Учебные вопросы

1. Структура Bash-сценария, переменные, quoting и command substitution
2. Аргументы \$1, \$#, \$?, "\$@"
3. if, case, простой for и функции
4. Exit codes
5. Базовая обработка ошибок
6. Логирование и grep/awk
7. Безопасный повторный запуск

Главный результат лекции - рабочий health-check

server-health.sh проверяет состояние сервера

Основные проверки: заполнение диска и состояние systemd-службы

Сценарий выводит понятное сообщение для администратора

Сценарий возвращает exit code для мониторинга или cron

Сначала делаем простую рабочую версию, затем добавляем улучшения

1. Bash-сценарий начинается с понятного каркаса

```
#!/usr/bin/env bash  
SERVICE="nginx"  
MOUNTPPOINT="/"   
DISK_WARN=80  
DISK_CRIT=90  
main "$@"
```

Каркас сценария задает роли частей

variables → настройки сценария

functions → отдельные проверки

main → порядок выполнения

return → результат функции

exit → итоговый результат всего сценария

Глубокий strict mode не должен быть первым, что видит студент

Health Check только проверяет состояние

server-health.sh читает состояние сервера

Он не перезапускает службы автоматически

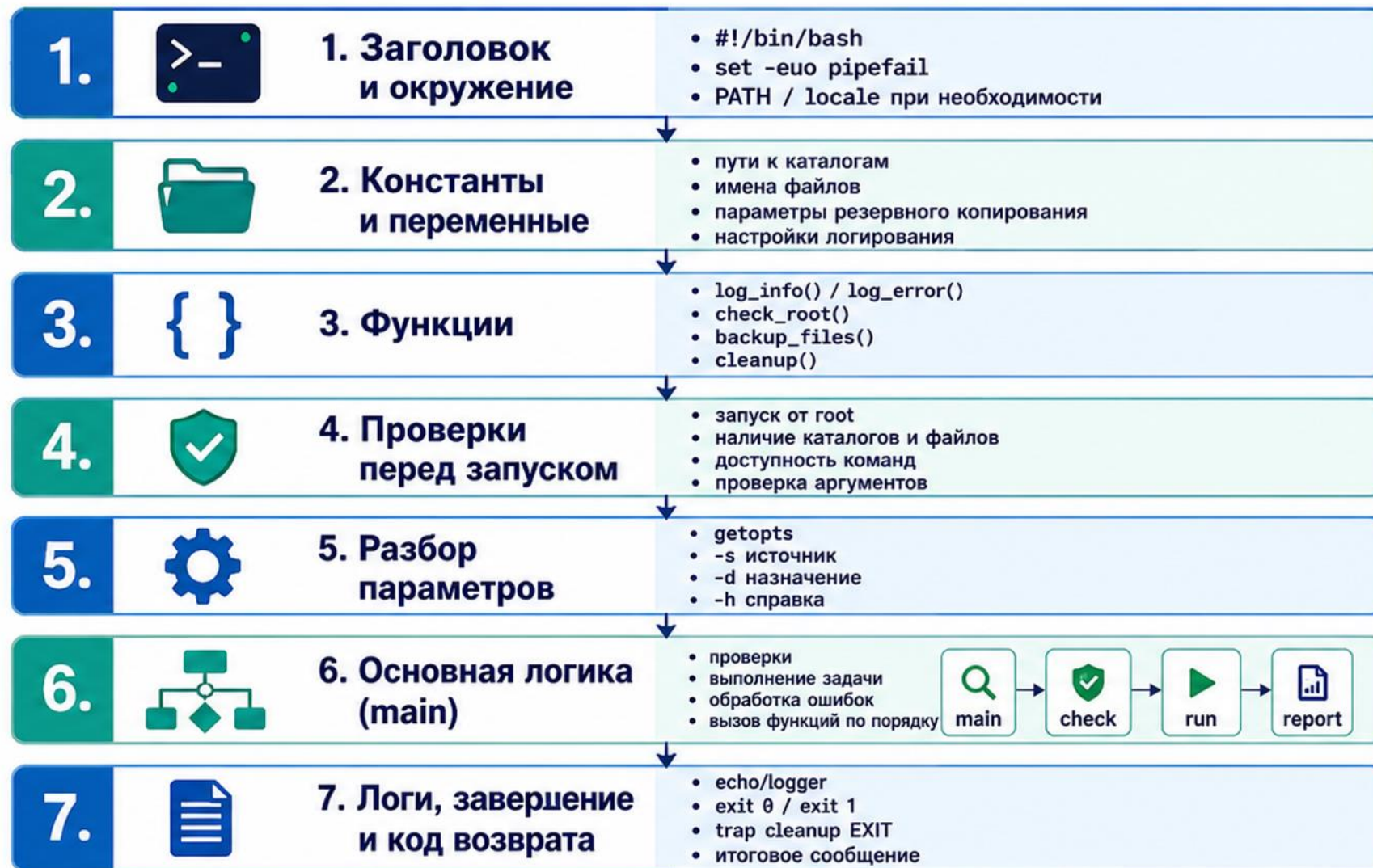
Он не удаляет файлы автоматически

Он не редактирует конфигурацию автоматически

Автоматическая проверка и автоматическое

исправление - разные задачи

Структура административного Bash-сценария



Принципы хорошего сценария



понятные имена



проверка ошибок



логирование действий



функции вместо длинного кода



предсказуемый код возврата



Общий шаблон



shebang



переменные



функции



проверки



разбор аргументов



main



exit

Переменные заменяют повторяющиеся значения

`SERVICE="nginx"` - имя проверяемой службы

`MOUNTPOINT="/"` - файловая система для проверки

`DISK_WARN=80` - порог предупреждения

`DISK_CRIT=90` - критический порог

Переменная делает сценарий читаемым и упрощает изменение настроек

После базового примера можно добавить `readonly`

`EXIT_OK=0` как улучшение

Quoting защищает сценарий от пробелов и сюрпризов shell

`cd "$path"` безопаснее, чем `cd $path`

`"$var"` - значение переменной подставляется

`'$var'` - остается буквальным текст `$var`

Почти всегда подставляем переменную в двойных кавычках

Это особенно важно для путей, имен служб и аргументов пользователя

Command substitution сохраняет вывод команды

`$(command)` выполняет команду и подставляет ее
ВЫВОД

`host=$(hostname -s)` сохраняет короткое имя узла

`used=$(df -P / | awk 'NR==2 {print $5}')` сохраняет Use% файловой системы

Command substitution удобно использовать для фактов, которые нужны дальше в сценарии

Не нужно делать multiline command substitution
центральной темой для первой версии

local ограничивает переменную внутри функции

local used объявляет переменную только внутри функции

Это снижает риск случайно изменить переменную из другой части сценария

Хорошая практика: объявить local, затем присвоить значение

Пример: local used

Следующая строка: `used=$(df -P "$MOUNTPOINT" | awk 'NR==2 {print $5}')`

2. Аргументы делают сценарий гибким

\$0 - имя сценария

\$1 - первый аргумент

\$# - количество аргументов

\$? - exit code предыдущей команды

SERVICE="\${1:-nginx}" задает nginx по умолчанию, если аргумент не передан

Пример запуска: ./server-health.sh nginx

"\$@" передает все аргументы корректно

"\$@" передает все аргументы, сохраняя их
отдельными значениями

main "\$@" передает аргументы из сценария в
функцию main

"\$*" объединяет аргументы иначе и чаще не подходит

Для junior достаточно запомнить правило: в main
передаем "\$@"

Аргументы пользователя нельзя склеивать случайно

getopts добавляют после базовых аргументов

Пример запуска: `./server-health.sh -s nginx -w 80 -c 90`

```
while getopts "hs:w:c:" opt; do
```

```
case "$opt" in
```

```
s) SERVICE="$OPTARG" ;;
```

```
w) DISK_WARN="$OPTARG" ;;
```

```
c) DISK_CRIT="$OPTARG" ;;
```

getopts не должен появляться раньше, чем студент
понял \$1 и "\$@"

Валидация отличает ошибку сценария от аварии сервера

`[[ "$DISK_WARN" =~ ^[0-9]+$ ]]` проверяет, что порог является числом

`(( DISK_WARN < DISK_CRIT ))` проверяет порядок порогов

Если пользователь ввел неверный параметр, это UNKNOWN

UNKNOWN означает ошибку проверки или ее параметров

CRITICAL означает, что проверяемый объект действительно в критическом состоянии

3. if - главная конструкция проверки

```
if systemctl is-active --quiet "$SERVICE"; then  
echo "OK"  
else  
echo "CRITICAL"  
fi
```

Команда внутри if сама определяет, какая ветка будет выполнена

Числовые условия удобнее писать через (())

```
if (( used >= DISK_CRIT )); then  
printf 'CRITICAL: disk=%s%%\n' "$used"  
elif (( used >= DISK_WARN )); then  
printf 'WARNING: disk=%s%%\n' "$used"  
fi
```

Для чисел (()) читается понятнее, чем строковые
сравнения

case выбирает действие по одному значению

case удобен в getopt

h) usage ;; показывает справку

s) SERVICE="\$OPTARG" ;; задает службу

w) DISK_WARN="\$OPTARG" ;; задает warning-порог

c) DISK_CRIT="\$OPTARG" ;; задает critical-порог

Не нужно превращать case в сложную state-machine

for подходит для заранее известного списка

```
for svc in nginx ssh; do  
systemctl is-active "$svc"  
done
```

Так можно проверить несколько известных служб

Дополнительно: SERVICES=(nginx ssh)

Массивы полезны, но не являются обязательным блоком первой версии

Функция должна делать одну проверку

`check_disk` проверяет диск

`check_service` проверяет службу

Одна функция → одна проверка

`return` возвращает код функции

`exit` завершает весь сценарий

Такой сценарий проще читать, тестировать и расширять

check_service: полная функция проверки службы

```
check_service() {  
  local service="$1"  
  if systemctl is-active --quiet "$service"; then  
    printf 'OK: service %s is active\n' "$service"; return 0  
  fi  
  printf 'CRITICAL: service %s is inactive\n' "$service";  
  return 2  
}
```

check_disk: получаем процент заполнения

```
check_disk() {  
  local used  
  used=$(df -P "$MOUNTPOINT" | awk 'NR==2 {gsub(/%/,  
  "", $5); print $5}')
```

df -P дает предсказуемый формат вывода

awk берет вторую строку и поле Use%

gsub убирает знак %, чтобы значение можно было
сравнивать как число

check_disk: сравниваем с порогами

```
if (( used >= DISK_CRIT )); then return 2
```

```
elif (( used >= DISK_WARN )); then return 1
```

```
else return 0
```

printf выводит OK, WARNING или CRITICAL с текущим процентом

В основной лаборатории проверяем только

MOUNTPOINT="/"

tmpfs, squashfs, container mounts и network FS -
дополнительно

Схема: функции server-health.sh

Сценарий проверяет состояние сервера (диск и systemd-службу) и возвращает понятный exit code



ПРИНЦИПЫ



Модульность

(каждая функция решает одну задачу)



Читаемость

(понятные имена и логика)



Безопасность

(read-only проверки, нет изменений системы)



Надежность

(обработка ошибок, trap, предсказуемый exit code)



Расширяемость

(легко добавить новые проверки)

4. Exit codes связывают сценарий с мониторингом

0 → OK

1 → WARNING

2 → CRITICAL

3 → UNKNOWN

Это соглашение нашего monitoring script / Nagios-style plugins

Это не универсальное значение всех Linux exit codes

Несколько проверок агрегируются в один итог

Если есть CRITICAL → exit 2

Иначе если есть UNKNOWN → exit 3

Иначе если есть WARNING → exit 1

Иначе → exit 0

Так мониторинг получает один итоговый код

Порядок важнее простой числовой сортировки

Диаграмма: агрегация OK, WARNING, CRITICAL и UNKNOWN

Как server-health.sh объединяет результаты нескольких проверок в один итоговый статус



Ключевая идея

Сценарий возвращает один итоговый exit code для мониторинга, cron, Zabbix, Nagios/Icinga.

5. Базовая обработка ошибок начинается с ожидаемых ситуаций

Неактивная служба - не ошибка Bash, а результат проверки

Заполненный диск - не ошибка Bash, а состояние сервера

Неверный параметр DISK_WARN - ошибка проверки, поэтому UNKNOWN

Ожидаемые неуспешные проверки обрабатываются через if

Сценарий должен объяснять, что именно пошло не так

set -euo pipefail полезен, но не должен перегружать первую версию

set -u сообщает об обращении к неизвестной
переменной

pipefail делает ошибку внутри pipeline заметнее

set -e может остановить сценарий при
необработанной ошибке

Ожидаемые отказы все равно обрабатывают явно
через if

Для первой версии важнее понятная логика, затем
можно усилить режим выполнения

Дополнительно: trap и mktemp очищают временные ресурсы

```
tmpfile=$(mktemp)
```

```
trap 'rm -f "$tmpfile"' EXIT
```

trap выполняет команду при завершении сценария

mktemp безопасно создает временный файл

ERR trap и set -E не являются обязательной темой этой лекции

6. Логирование разделяет результат и ошибки выполнения

STDOUT - итог проверки для пользователя или мониторинга

STDERR - ошибка выполнения самого сценария

Journald - история запусков

```
logger -t server-health "disk usage=${used}%"
```

записывает сообщение в journal

```
journalctl -t server-health
```

показывает записи этого сценария

grep и awk нужны только там, где они помогают проверке

grep → найти строки

awk → разобрать поля

sort → отсортировать

uniq -c → посчитать одинаковые строки

sed → изменить текстовый поток

server-health.sh должен использовать только нужные утилиты, а не весь набор сразу

7. Идемпотентность делает повторный запуск безопасным

Идемпотентность: запустили 1 раз или 10 раз, система не ломается

`mkdir -p /some/dir` можно запускать повторно

`echo "line" >> file` может добавлять дубликаты

Read-only health-check проще сделать безопасным при повторном запуске

`server-health.sh` ничего не исправляет автоматически, поэтому риск ниже

Cron запускает сценарий в другом окружении

Работает в терминале ≠ обязательно работает в cron

```
PATH='/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin'
```

```
export PATH
```

В cron лучше использовать абсолютные пути к файлам

Сценарий не должен зависеть от интерактивного окружения пользователя

sudo и flock оставляем как важные ограничения

Автоматический сценарий не должен зависеть от интерактивного запроса пароля sudo

Сценарий запускают сразу от нужного пользователя

Дополнительно: flock защищает от параллельного запуска

flock полезен, если cron может запустить второй экземпляр до завершения первого

Для базовой лаборатории flock не является обязательной частью server-health.sh

Сценарий проверяют до запуска на сервере

bash -n server-health.sh проверяет синтаксис Bash
shellcheck server-health.sh рекомендует улучшения и
предупреждает о типовых ошибках

После syntax check проверяют разные состояния

Нужно отдельно проверить OK, WARNING, CRITICAL и
UNKNOWN

Тестирование сценария важнее уверенности, что код
выглядит правильно

Версия 1: базовый server-health.sh

Использует variables: SERVICE, MOUNTPOINT,
DISK_WARN, DISK_CRIT

Использует functions: check_disk и check_service

Использует if для проверки службы и диска

Использует return внутри функций

Использует exit в конце main

Студент сначала должен полностью понять эту версию

Версия 1: порядок main

```
main() {  
  check_disk; disk_status=$?  
  check_service "$SERVICE"; service_status=$?  
  агрегировать disk_status и service_status  
  exit "$result"  
}
```

Версия 2: улучшенный server-health.sh

getopts добавляет -s, -w, -c и -h

validation проверяет числовые пороги и их порядок

UNKNOWN отделяет ошибку параметров от аварии сервера

logger отправляет краткое событие в journald

RATH фиксирует окружение для cron

optional strict mode добавляется после понимания базовой версии

Лабораторная приемка проверяет поведение, а не только код

```
bash -n server-health.sh
```

```
./server-health.sh
```

```
echo $?
```

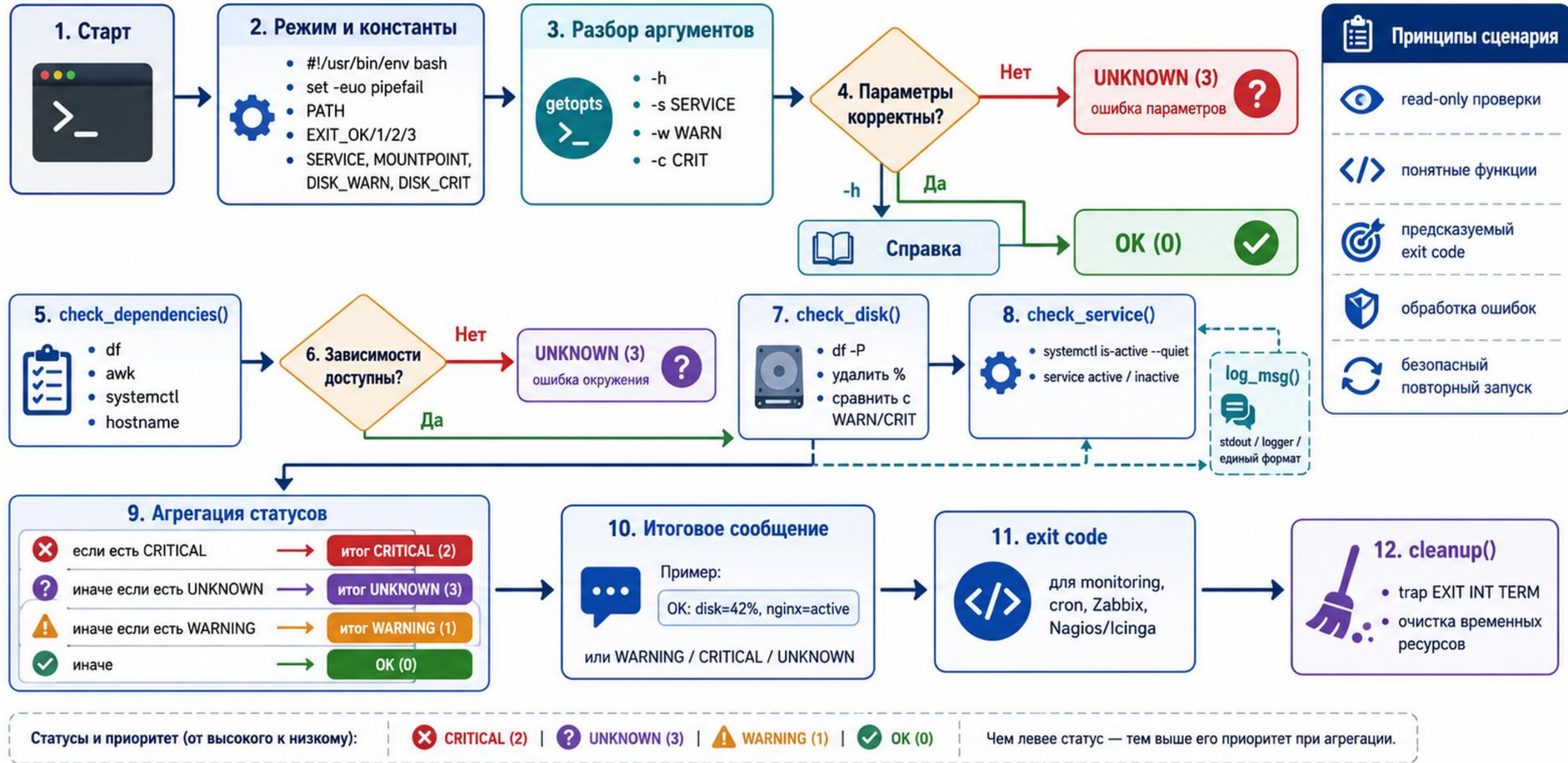
```
./server-health.sh nonexistent.service
```

Проверить WARNING и CRITICAL безопасно через
учебные пороги

Подтвердить OK, WARNING, CRITICAL и UNKNOWN

Диаграмма: логика server-health.sh

Как сценарий проверяет сервер и возвращает итоговый статус



Итоги лекции

Административный Bash-сценарий должен быть понятным, проверяемым и предсказуемым

Переменные, quoting, command substitution и функции дают базовый каркас

if, case и for закрывают основные управляющие конструкции junior-уровня

Exit codes 0, 1, 2 и 3 позволяют использовать сценарий в мониторинге

server-health.sh сначала должен только проверять, а не автоматически исправлять

Продвинутые паттерны добавляются после рабочей

Контрольные вопросы

1. Для чего администратору Bash-сценарии?
2. Почему переменные обычно используют в двойных кавычках?
3. Что делает `$(command)`?
4. Что содержат `$1` и `$?`?
5. Для чего нужен `if`?
6. Чем `return` отличается от `exit`?
7. Что означает `exit code 0`?
8. Как проверить `systemd`-службу из Bash?
9. Как проверить синтаксис сценария?
10. Почему `health-check` не должен автоматически